

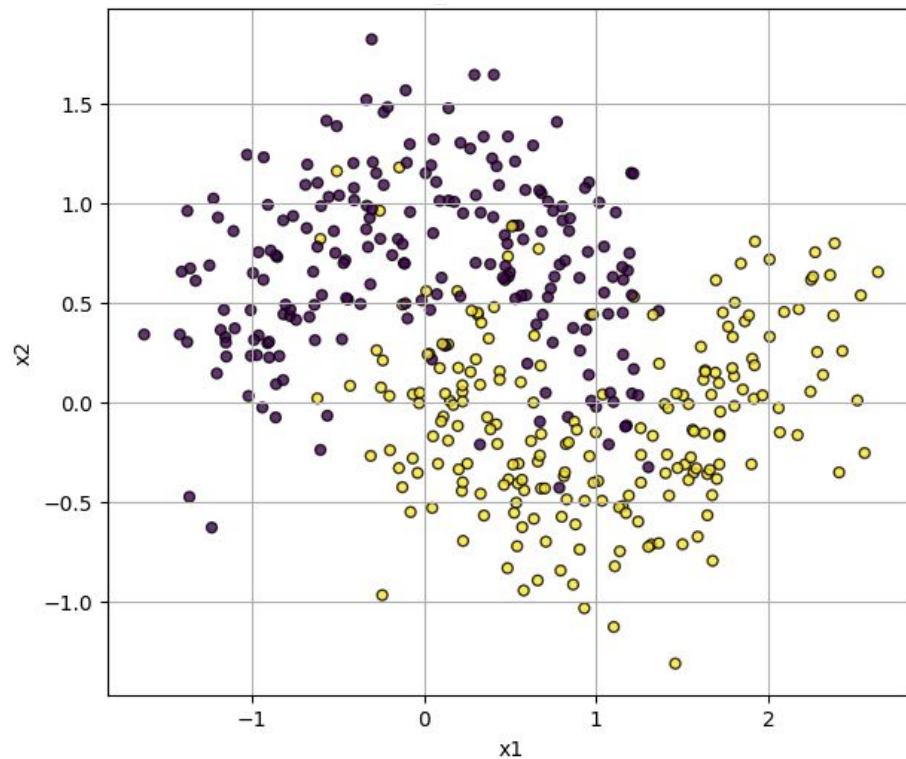
Más árboles

Clase práctica 6

Motivación

- Ya vimos que los cortes en un árbol de decisión son paralelos y **no parecen muy “flexibles”**, podemos suavizarlos?
- Hay alguna técnica que nos ayude a **“aprovechar”** más nuestro único training set?

Dataset Ejemplo

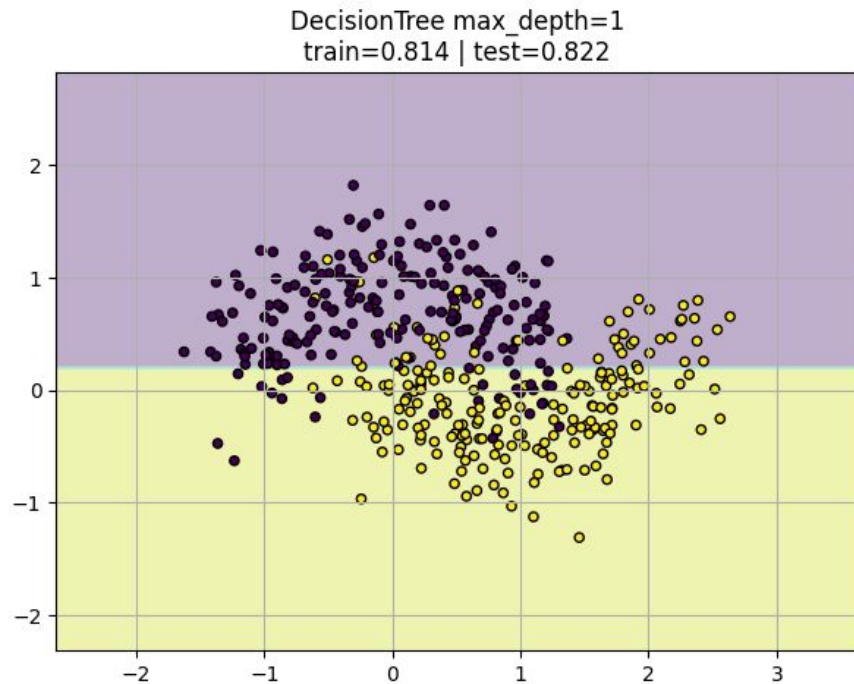


Problema de Clasificación

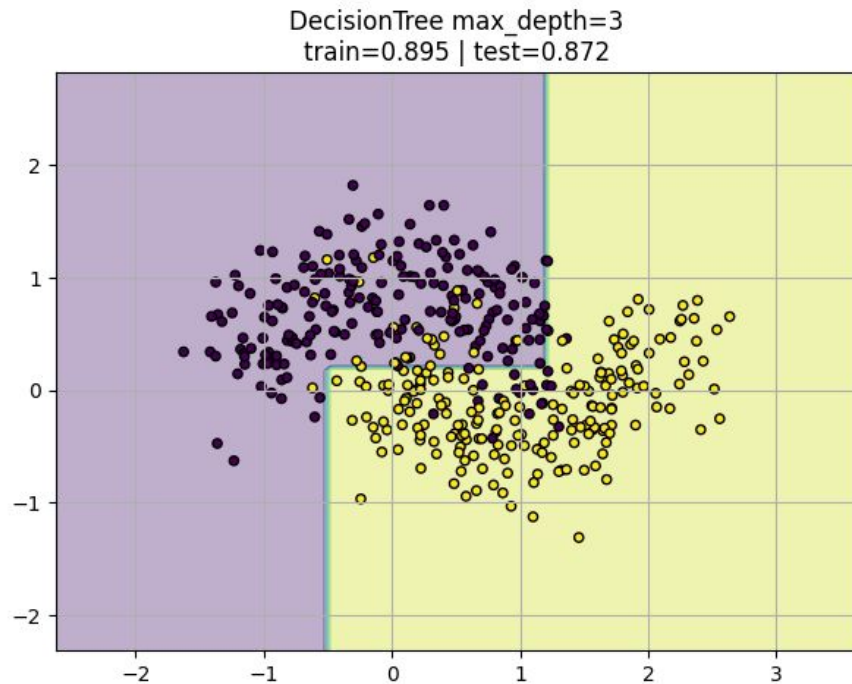
Queremos en base a **x1** y **x2** predecir si la observación es **amarillo** o **violeta**. Vamos a utilizar como modelo árboles de decisión y ver como ciertas técnicas afectan los resultados.

Dado que estos datos asumimos que estan completamente balanceados, es decir, 50% es **amarillo** y 50% es **violeta**, entonces podemos usar como métrica didáctica el **accuracy**. Valor que vamos a evaluar en **train y test**.

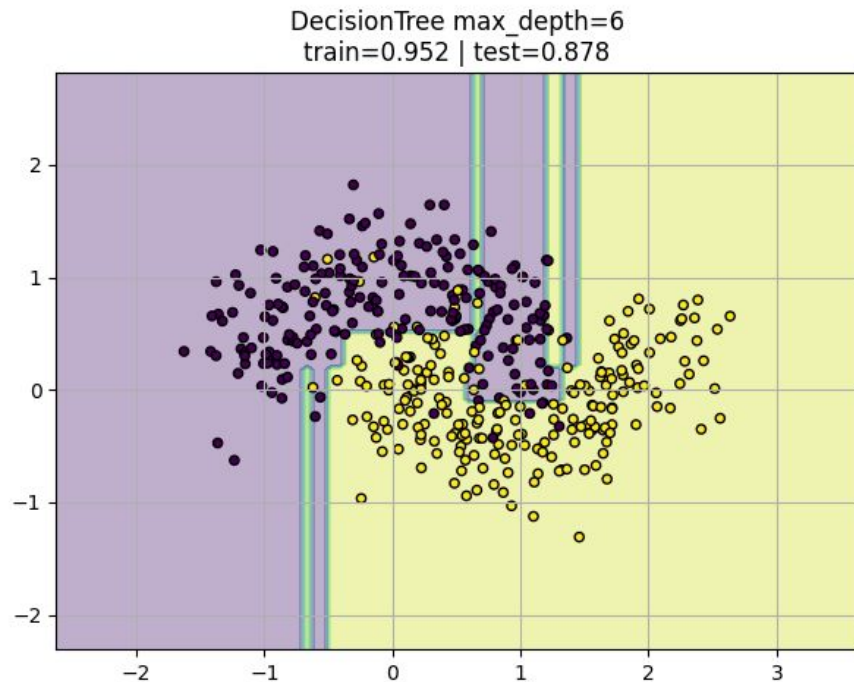
Frontera con max_depth = 1



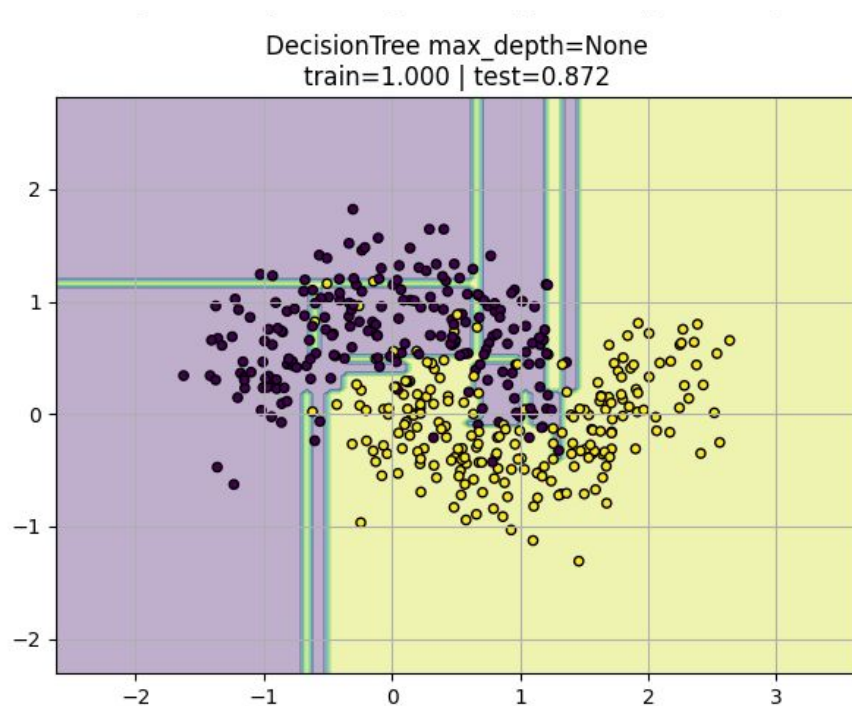
Frontera con max_depth = 3



Frontera con max_depth = 6



Frontera con max_depth = none

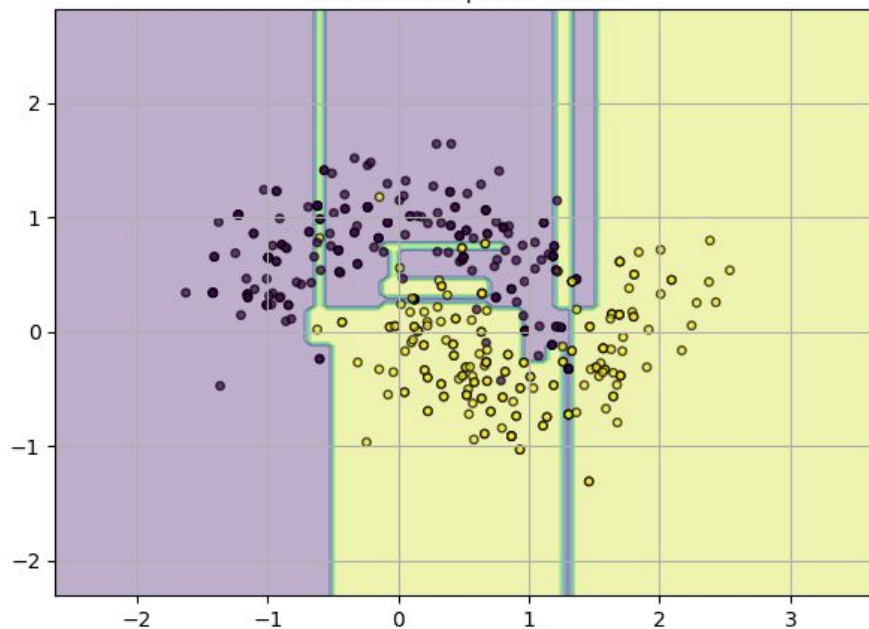


Preguntémonos

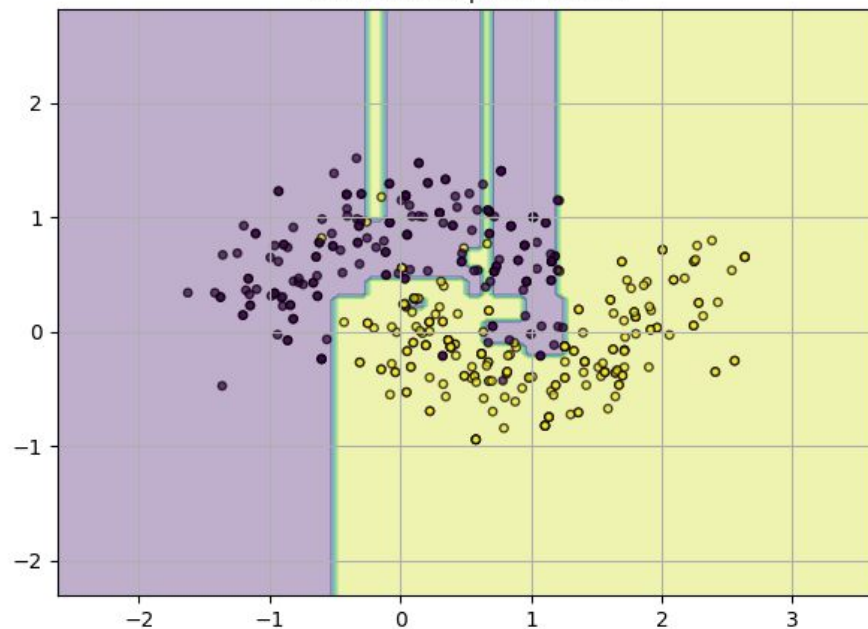
- Qué pasa con la performance en train y test a medida que cambia el hiperparámetro de `max_depth`?
- Hay algo en los plots que te llama la atención acerca de los árboles con mayor profundidad?

Bootstrapping - max_depth = none

Bootstrap sample #1
train=0.950 | test=0.839

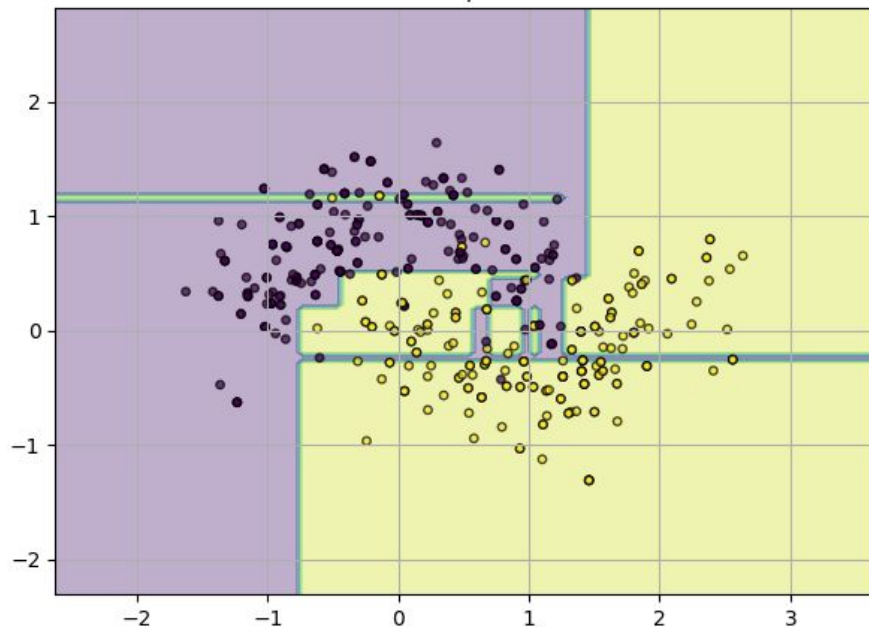


Bootstrap sample #2
train=0.952 | test=0.828

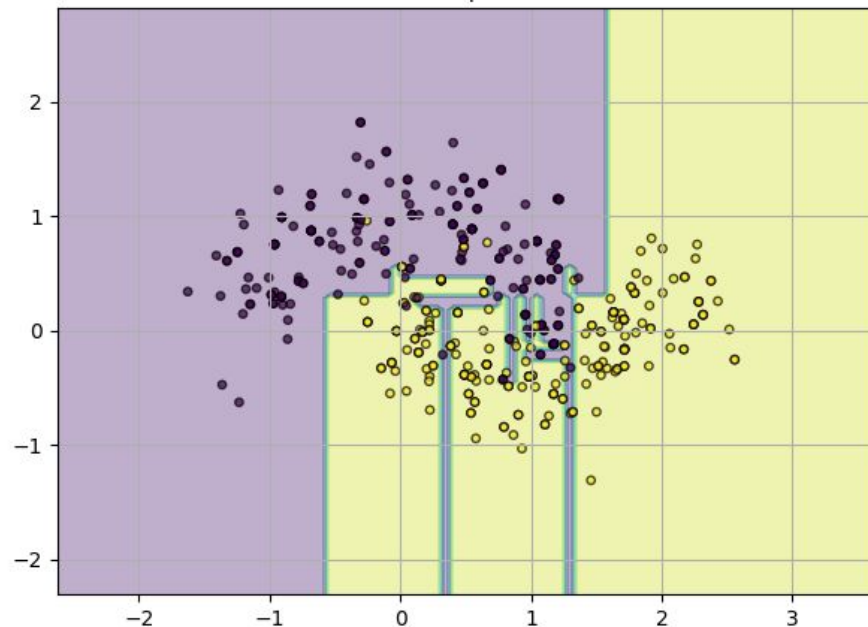


Bootstrapping - max_depth = none

Bootstrap sample #3
train=0.943 | test=0.844



Bootstrap sample #4
train=0.940 | test=0.872



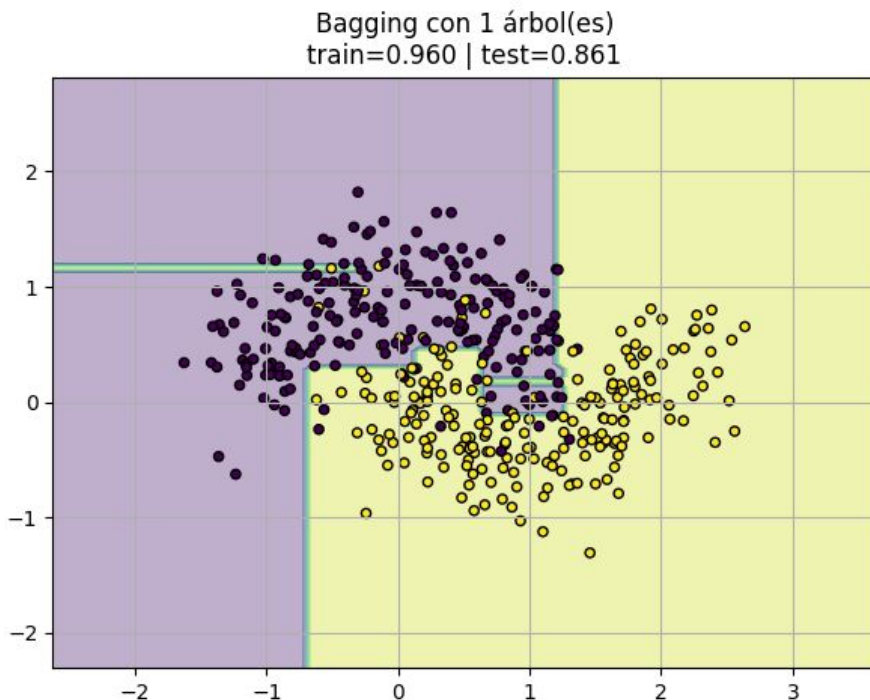
Bootstrapping - max_depth = none

- Cada bootstrap y árbol fiteado en el mismo parece tener **peor performance en test** que antes cuando entrenamos con el dataset completo. Pueden inferir por qué?

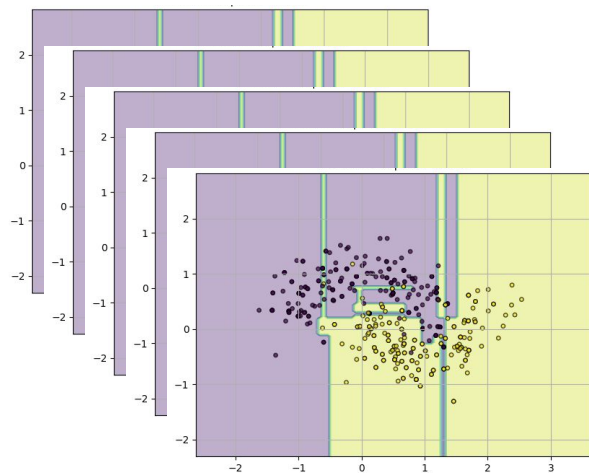
Y si repetimos este proceso lo suficiente?

Bagging con 1 árbol es simplemente, bootstrappar (resamplear mi dataset) y fitear un árbol.

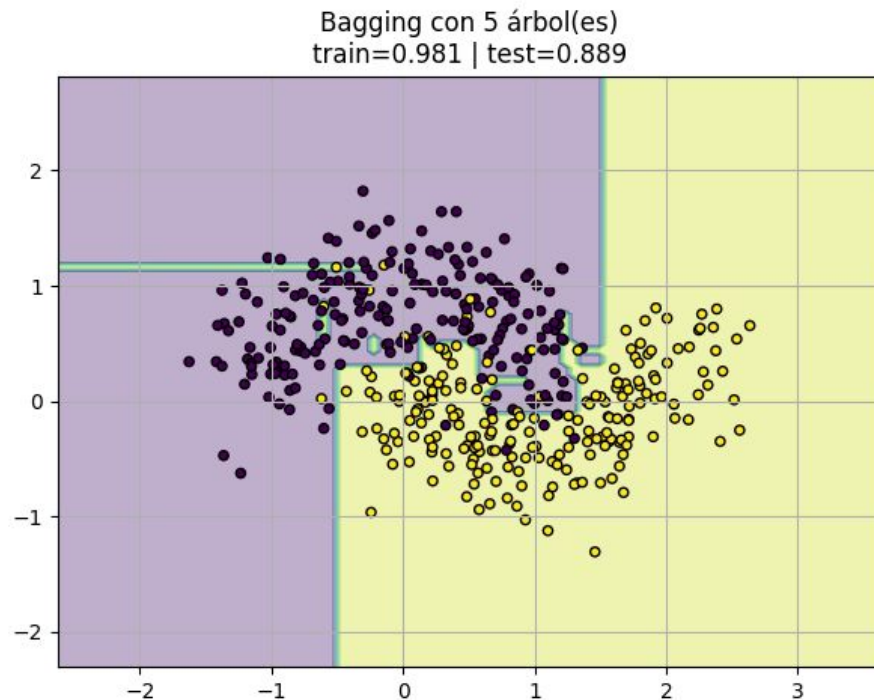
No hay agregación y encima resampleé mi data...



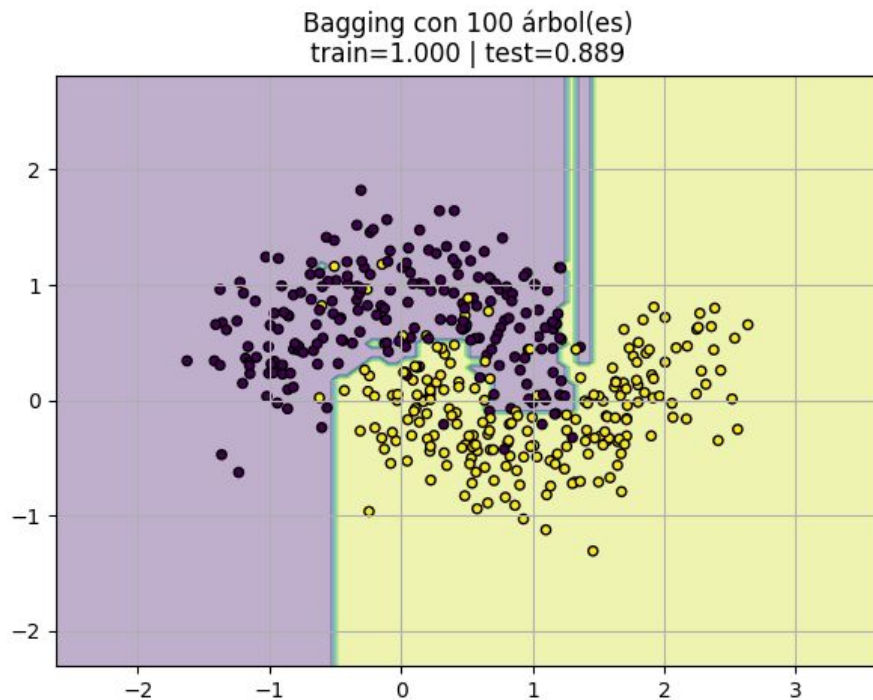
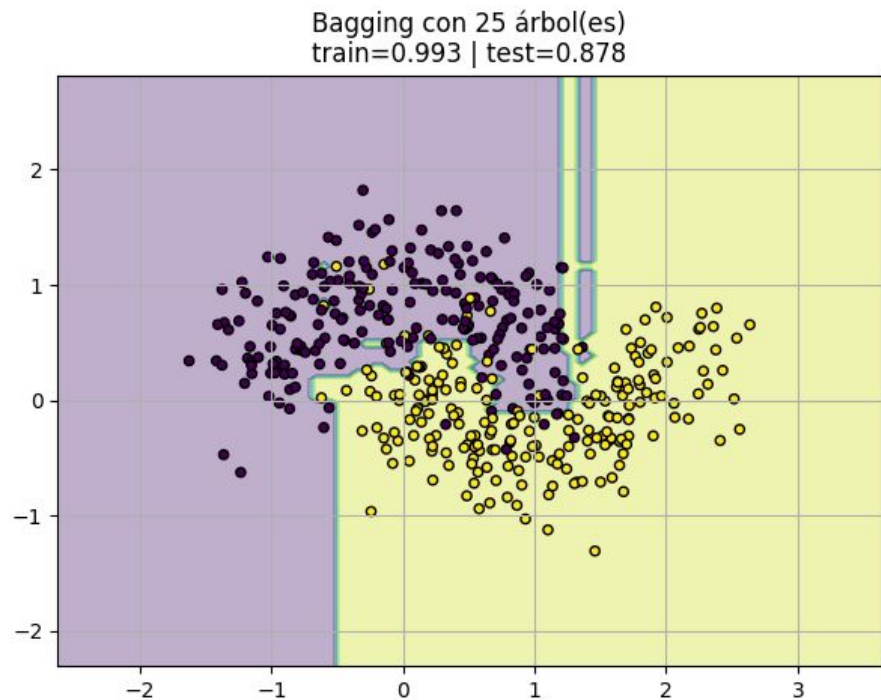
Bagging - max_depth = none



5 bootstraps y “modelos independientes”



Bagging - max_depth = none



Conclusiones

- Cuando bootstrapeamos obtenemos una muestra en base a la distribución de la original y fiteamos el modelo, esto nos dá diferentes datasets de train y diferentes modelos resultantes.
- Agregar estos modelos nos da fronteras más suaves, y que nos hacen más robustos ante distintos dataset. Dependemos menos de un solo árbol, baja la varianza.
- Random Forest hace lo mismo para las columnas, más robustez aún.

Búsqueda de hiperparámetros

Motivación

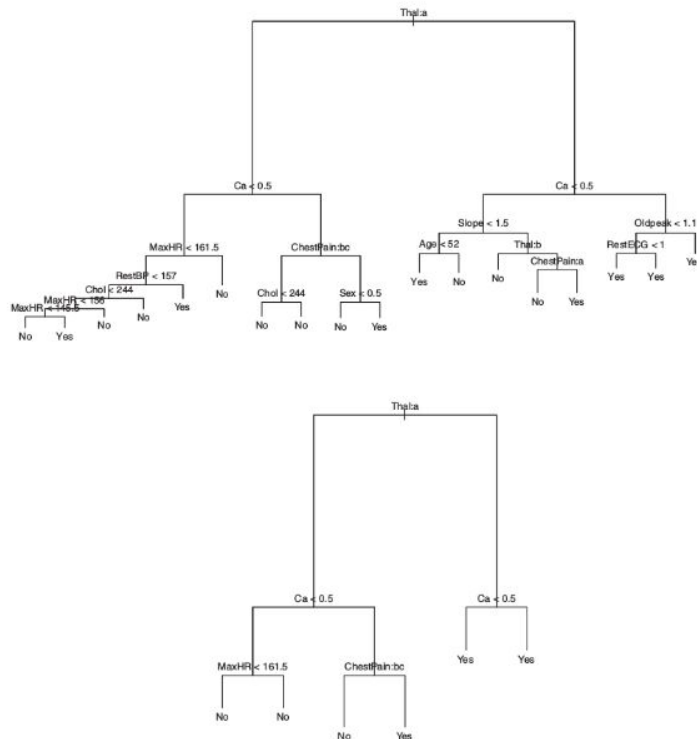
- ¿Cómo podemos encontrar los hiperparámetros que **optimizan** nuestro modelo?
- ¿Cuáles son las **ventajas y desventajas** de esas estrategias?

Enfoque hasta ahora | Un hiperparámetro

Ejemplo: **altura máxima** de un árbol.

¿Qué **altura** nos conviene usar?

Hasta ahora, nuestro enfoque fue **elegir valores y probarlos** para el hiperparámetro y analizar la *performance* del modelo en validación.

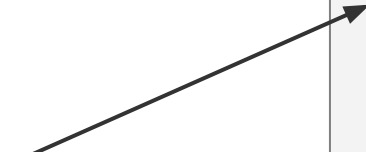


Enfoque hasta ahora | N hiperparámetros

Queremos encontrar los valores de esos hiperparámetros que optimicen el modelo en validación.

¿Cómo lo venimos haciendo intuitivamente?

¡Se llama **grid search**!

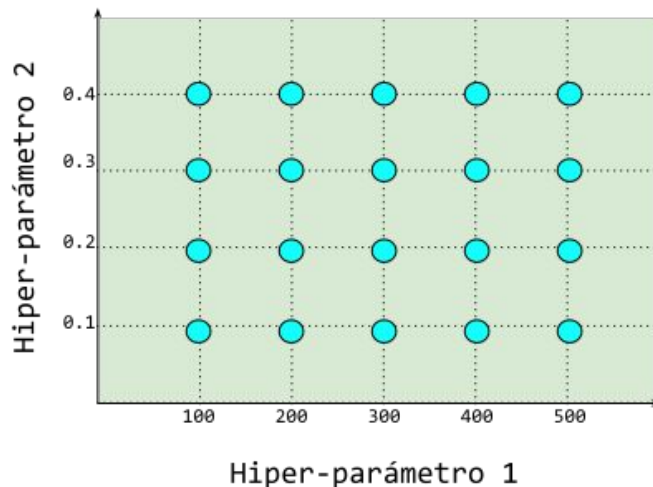


```
for val_1 in valores_hip_1:
    ...
    for val_N in valores_hip_N:
        train(val_1 , ..., val_N)
        calcular_score()

        if score mejoró:
            guardar_valores()
```

Grid search

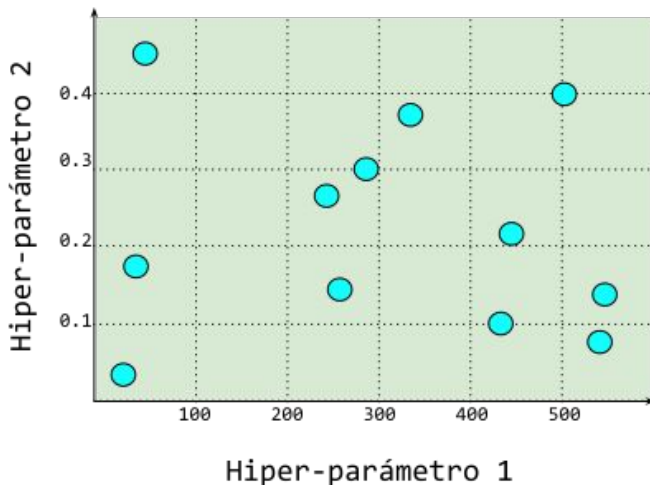
- Dado un conjunto de hiperparámetros y posibles valores seleccionados, se prueban todas las **combinaciones**.
- Se definen **explícitamente** los posibles valores de cada hiperparámetro.
- Requiere **mucho cómputo**.



```
sklearn.model_selection.GridSearchCV()
```

Random search

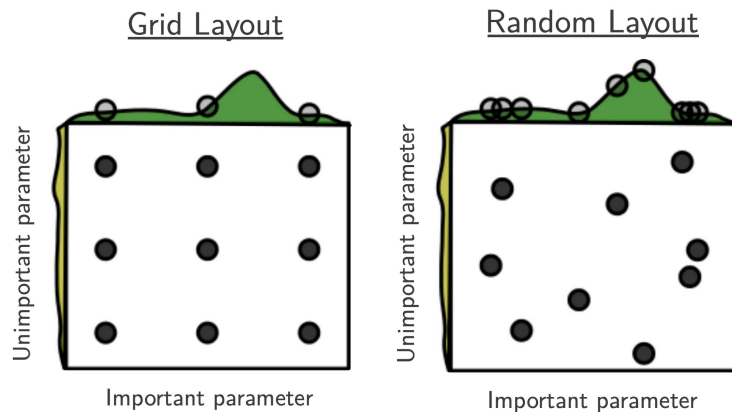
- Dado un espacio de hiperparámetros definido, se toman **combinaciones aleatorias**.
- Se definen valores a probar de los hiperparámetros o **distribuciones** de las cuales se seleccionarán los mismos (útil para valores continuos).
- Requiere **menos cómputo**.
- Se define el número de **iteraciones** máximas.
- **No tiene memoria** de la performance de iteraciones pasadas (grid tampoco).



```
sklearn.model_selection.RandomizedSearchCV()
```

¿Cuál nos conviene usar?

- Evidencia empírica muestra que **random search** encuentra configuraciones iguales o, a veces, incluso mejores que grid search, en una fracción del tiempo.
- Random search permite **explorar** el espacio de hiperparámetros de manera **más eficiente**.
- En consecuencia, si se asigna el **mismo tiempo** de cómputo, random search encuentra **mejores configuraciones**.



Grid y random search | Modo

Conozcamos las implementaciones de grid search y random search en scikit-learn. Para ello, veamos `td6-p06-c-grid-random.ipynb`.

Grid y random search | Modo

```
tree = DecisionTreeClassifier(random_state = 22) # Creamos la instancia.  
scores = cross_val_score(tree, X, y, cv = KFold(4)) # Ejecutamos la validacion cruzada con 4 folds.  
  
print(f'Score: {scores.mean():.2f}.')
```

Score: 0.83.

Grid y random search | Modo

```
tree = DecisionTreeClassifier(random_state = 22) # Creamos la instancia.  
scores = cross_val_score(tree, X, y, cv = KFold(4)) # Ejecutamos la validacion cruzada con 4 folds.  
  
print(f'Score: {scores.mean():.2f}.')
```

Score: 0.83.

¿Qué hiperparámetros podemos tunear?

- criterion: {gini, entropy, log_loss};
- splitter: {best, random};
- max_depth: {5, 10, 20, 30, 40};
- min_samples_split: {2, 5, 10, 20};
- min_samples_leaf: {1, 2, 5, 10, 15} y
- min_impurity_decrease: {0, 0.05, 0.1}.

Grid y random search | Modo

```
tree = DecisionTreeClassifier(random_state = 22) # Creamos la instancia.
scores = cross_val_score(tree, X, y, cv = KFold(4)) # Ejecutamos la validacion cruzada con 4 folds.

print(f'Score: {scores.mean():.2f}.')
```

Score: 0.83.

¿Qué hiperparámetros podemos tunear?

- criterion: {gini, entropy, log_loss};
- splitter: {best, random};
- max_depth: {5, 10, 20, 30, 40};
- min_samples_split: {2, 5, 10, 20};
- min_samples_leaf: {1, 2, 5, 10, 15} y
- min_impurity_decrease: {0, 0.05, 0.1}.

Grid y random search | Modo

Definimos la búsqueda

```
from sklearn.model_selection import GridSearchCV

parameters = {'criterion': ('gini', 'entropy', 'log_loss'),
              'splitter': ('best', 'random'),
              'max_depth': (5, 10, 20, 30, 40),
              'min_samples_split': (2, 5, 10, 20),
              'min_samples_leaf': (1, 2, 5, 10, 15),
              'min_impurity_decrease': (0, 0.05, 0.1)
              }

# GridSearchCV recibe un estimador que debe tener implementado el método score().
# Se definen los (hiper)parámetros a combinar.
# Se pueden definir los k-folds para cross-validation. Por defecto, cv = 5.
gs = GridSearchCV(estimator = DecisionTreeClassifier(random_state = 22),
                  param_grid = parameters,
                  cv = KFold(4))
```

Grid y random search | Modo

```
print(gs.best_score_)  
print(gs.best_params_)
```

```
0.8469565217391305
```

```
{'criterion': 'entropy', 'max_depth': 20, 'min_impurity_decrease': 0, 'min_samples_leaf': 5, 'min_samples_split': 20, 'splitter': 'random'}
```

Grid y random search | Modo

```
print(gs.best_score_)  
print(gs.best_params_)
```

0.8469565217391305

{'criterion': 'entropy', 'max_depth': 20, 'min_impurity_decrease': 0, 'min_samples_leaf': 5, 'min_samples_split': 20, 'splitter': 'random'}

```
best_tree = gs.best_estimator_  
best_tree
```

```
▼ DecisionTreeClassifier ⓘ ⓘ  
DecisionTreeClassifier(criterion='entropy', max_depth=20,  
                        min_impurity_decrease=0, min_samples_leaf=5,  
                        min_samples_split=20, random_state=22,  
                        splitter='random')
```

Accedemos al modelo con
los mejores hiperparámetros

Grid y random search | Documentación oficial

- `sklearn.model_selection.GridSearchCV()`: [enlace](#).
- `sklearn.model_selection.RandomizedSearchCV()`: [enlace](#).

¿Se pueden mejorar? 🤔

Con grid y random search, a medida que se exploran los hiperparámetros, se encuentran puntos del espacio que generan distintos resultados. Sin embargo, esas dos estrategias **no aprovechan el conocimiento que se va adquiriendo** a medida que se prueban distintos valores.

Bayesian optimization | Idea

Tratar la búsqueda de hiperparámetros como un **problema de machine learning** en sí mismo.

- Asumamos que existe una **función** f que, para cada **vector de hiperparámetros** x , devuelve la **performance estimada del modelo** (p).
- Desconocemos la forma de f , pero la podemos **estimar** (si bien es caro hacerlo).
- Cuando **probamos un nuevo valor** de x , adquirimos más información que permite estimar mejor f (es decir, adquirimos un dato más para estimar f).
- Dada una estimación de f , ¿en **dónde** probamos un nuevo x ? Dos objetivos:
 - **Exploration**: probar nuevos y diversos valores de x para estimar mejor f **en todo el espacio**; el óptimo quizás está en una zona aún no explorada.
 - **Exploitation**: dada nuestra estimación de f en un momento, probar valores **cercanos al óptimo provisorio**, de acuerdo a nuestra estimación actual, e intentar aprender más de f en esa área prometedora.

Optimización bayesiana **balancea** el *trade-off* entre esos dos objetivos, con el fin de **optimizar** p .

Bayesian optimization | Modo

Conozcamos una implementación de bayesian optimization con la librería **HyperOpt**. Para ello, veamos `td6-p06-d-bayesian.ipynb`.

Bayesian optimization | Modo

```
from hyperopt import hp

space = {'criterion': hp.choice('criterion', ['gini', 'entropy', 'log_loss']),
        'splitter': hp.choice('splitter', ['best', 'random']),
        'max_depth': hp.uniformint('max_depth', 3, 80),
        'min_samples_split': hp.uniformint('min_samples_split', 2, 20),
        'min_samples_leaf': hp.uniformint('min_samples_leaf', 1, 20),
        'min_impurity_decrease': hp.uniform('min_impurity_decrease', 0, 0.1)
    }
```

Bayesian optimization | Modo

```
from hyperopt import hp

space = {'criterion': hp.choice('criterion', ['gini', 'entropy', 'log_loss']),
        'splitter': hp.choice('splitter', ['best', 'random']),
        'max_depth': hp.uniformint('max_depth', 3, 80),
        'min_samples_split': hp.uniformint('min_samples_split', 2, 20),
        'min_samples_leaf': hp.uniformint('min_samples_leaf', 1, 20),
        'min_impurity_decrease': hp.uniform('min_impurity_decrease', 0, 0.1)}
}
```

Definimos una función de costo

```
def objective(params):
    tree = DecisionTreeClassifier(**params, random_state = 22)
    score = cross_val_score(tree, X, y, cv = KFold(4)).mean() # Aplicamos validación cruzada con 4 folds.
    return {'loss': 1 - score, 'status': STATUS_OK}
```

Bayesian optimization | Modo

```
best = fmin(objective,  
            space,  
            algo = tpe.suggest,  
            max_evals = 1800,  
            rstate = np.random.default_rng(22))
```

El score resultante equivale a 0,866 aproximadamente.

Ese valor de score es mayor al $\approx 0,847$ que obtuvimos antes con *grid search* y al $\approx 0,859$ que obtuvimos antes con *random search*.

Bayesian optimization | Modo

```
best = fmin(objective,  
            space,  
            algo = tpe.suggest,  
            max_evals = 1800,  
            rstate = np.random.default_rng(22))
```

El score resultante equivale a 0,866 aproximadamente.

Ese valor de score es mayor al $\approx 0,847$ que obtuvimos antes con *grid search* y al $\approx 0,859$ que obtuvimos antes con *random search*.

Bayesian Optimization puede encontrar mejores resultados en un menor tiempo

Bayesian optimization | Documentación oficial

- Librería **HyperOpt**: [enlace](#).
- **scikit-optimize** como librería alternativa: [enlace](#).